03_time_travel

August 18, 2026

1 NB3 — Time Travel + MERGE Upsert (lightweight)

Maps to slide §3 + deliverable bullet 3.

> Spark equivalent: `option("versionAsOf", N)` `DeltaTable(path, version=N)` `MERGE INTO ...` `dt.merge(source, predicate)`

[1]:
```
import _setup  # noqa: F401  -- adds scripts/ to sys.path
import time
import polars as pl
from deltalake import DeltaTable, write_deltalake
from lakehouse import path, reset

table_path = path("scratch", "customers_tt")
reset(table_path)
```

1.1 1. Build version history

v0: initial 100K · v1: schema add · v2: MERGE upsert · v3: bad data

[2]:
```
# v0 - initial load
v0 = pl.DataFrame({
    "customer_id": list(range(100_000)),
    "status":      ["active"] * 100_000,
    "score":       [i % 1000 for i in range(100_000)],
})
write_deltalake(table_path, v0.to_arrow(), mode="overwrite")

# v1 - add `tier` column (schema evolution)
v1 = (pl.from_arrow(DeltaTable(table_path).to_pyarrow_table())
        .with_columns(
            pl.when(pl.col("score") > 800).then(pl.lit("gold")).otherwise(pl.
 ↪lit("silver")).alias("tier")
        ))
write_deltalake(table_path, v1.to_arrow(), mode="overwrite",␣
 ↪schema_mode="overwrite")

# v2 - MERGE upsert 100K (50K updates, 50K inserts)
updates = pl.DataFrame({
```

```
    "customer_id": list(range(50_000, 150_000)),
    "status":      ["vip"] * 100_000,
    "score":       [999] * 100_000,
    "tier":        ["platinum"] * 100_000,
})
t0 = time.time()
(DeltaTable(table_path)
    .merge(source=updates.to_arrow(),
           predicate="t.customer_id = s.customer_id",
           source_alias="s", target_alias="t")
    .when_matched_update_all()
    .when_not_matched_insert_all()
    .execute())
print(f"MERGE 100K rows: {time.time()-t0:.2f}s")

# v3 - simulate bad data
bad = pl.DataFrame({
    "customer_id": list(range(50)),
    "status":      [None] * 50,
    "score":       [-1] * 50,
    "tier":        ["UNKNOWN"] * 50,
}, schema={"customer_id": pl.Int64, "status": pl.Utf8, "score": pl.Int64,␣
 ↪"tier": pl.Utf8})
write_deltalake(table_path, bad.to_arrow(), mode="append")
```

```
MERGE 100K rows: 0.09s
```

1.2 2. history() — audit trail

[3]:
```
for h in DeltaTable(table_path).history():
    print(f"  v{h['version']:>2}  {h['operation']:<25}  metrics={h.
 ↪get('operationMetrics', {})}")
```

```
  v 3  WRITE                     metrics={'execution_time_ms': 1,
'num_added_files': 1, 'num_added_rows': 50, 'num_partitions': 0,
'num_removed_files': 0}
  v 2  MERGE                     metrics={'execution_time_ms': 66,
'num_output_rows': 150000, 'num_source_rows': 100000, 'num_target_files_added':
1, 'num_target_files_removed': 1, 'num_target_files_scanned': 1,
'num_target_files_skipped_during_scan': 0, 'num_target_rows_copied': 50000,
'num_target_rows_deleted': 0, 'num_target_rows_inserted': 50000,
'num_target_rows_updated': 50000, 'rewrite_time_ms': 6, 'scan_time_ms': 50}
  v 1  WRITE                     metrics={'execution_time_ms': 12,
'num_added_files': 1, 'num_added_rows': 100000, 'num_partitions': 0,
'num_removed_files': 1}
  v 0  WRITE                     metrics={'execution_time_ms': 13,
'num_added_files': 1, 'num_added_rows': 100000, 'num_partitions': 0,
'num_removed_files': 0}
```

1.3 3. Time-travel queries

[4]:
```
v0_count = DeltaTable(table_path, version=0).to_pyarrow_table().num_rows
v1_cols  = DeltaTable(table_path, version=1).schema().to_arrow().names
print(f"v0 row count: {v0_count}")
print(f"v1 schema:    {v1_cols}")
```

```
v0 row count: 100000
v1 schema:    ['customer_id', 'status', 'score', 'tier']
```

1.4 4. RESTORE bad version (rollback)

`restore(2)` rewinds the *current* state of the table to whatever it was at version 2, recorded as a new version (v4). The old versions stay in history — restore is itself a transaction, fully auditable.

[5]:
```
t0 = time.time()
dt = DeltaTable(table_path)
dt.restore(2)
print(f"RESTORE → v2: {time.time()-t0:.2f}s   (target < 30s)")

# Verify the bad rows are gone - use delta-rs's native filter pushdown.
# (DuckDB's delta extension as of 1.5.x is stricter about post-RESTORE
# protocol entries than delta-rs writes; routing through delta-rs end-to-end
# avoids the InvalidProtocolError race.)
dt_after = DeltaTable(table_path)
bad_count = dt_after.to_pyarrow_table(filters=[("score", "<", 0)]).num_rows
print(f"Rows with score<0 after restore: {bad_count}  (expected 0)")
```

```
RESTORE → v2: 0.02s   (target < 30s)
Rows with score<0 after restore: 0  (expected 0)
```

1.5 5. history() — final audit trail (now includes the RESTORE)

[6]:
```
final_history = DeltaTable(table_path).history()
for h in final_history:
    print(f"  v{h['version']:>2}  {h['operation']:<25}")
print(f"\nTotal versions: {len(final_history)}  (target  5)")
```

```
  v 4  RESTORE
  v 3  WRITE
  v 2  MERGE
  v 1  WRITE
  v 0  WRITE

Total versions: 5  (target  5)
```

1.6 Deliverable check

- ☐ history() shows 5 versions (incl. RESTORE itself)
- ☐ MERGE 100K finished in < 60s (likely < 1s on lightweight path)

☐ RESTORE finished in < 30s and removed bad rows

[7]:
```
ops = [h["operation"] for h in final_history]
checks = {
    "history  5 versions":          len(final_history) >= 5,
    "history includes the RESTORE":  any("RESTORE" in o.upper() for o in ops),
    "MERGE recorded in history":     any("MERGE" in o.upper() for o in ops),
    "bad rows gone after restore":   bad_count == 0,
}
for k, v in checks.items():
    print(f"  [{'PASS' if v else 'FAIL'}] {k}")
assert all(checks.values()), "NB3 incomplete - see FAIL rows above"
print("\nNB3 complete.")
```

```
  [PASS] history  5 versions
  [PASS] history includes the RESTORE
  [PASS] MERGE recorded in history
  [PASS] bad rows gone after restore

NB3 complete.
```